

Anexo 21 — Entrenamiento multi-tarea (exploratorio)

Archivo: Scripts/train_multitask.py

Para que sirvió:

Entrenamiento de un entorno que intentaba combinar cabeza + saludo + pinza en una sola policy. Línea de trabajo que se reemplaza por políticas especializadas separadas. No formo parte del pipeline final; se incluye por completitud.

```
1 | """Entrenamiento PPO sobre DUMMultitaskEnv (v14: head + wave + grip).
2 |
3 | Hace warm-start opcional desde la policy v13 (final.zip) copiando solo los pesos
4 | con shape compatible. Esto reusa el "saber mirar" ya entrenado en v13.
5 |
6 | Uso:
7 |     # Smoke (50k steps, 8 envs, ~5 min): valida pipeline.
8 |     python Scripts/train_multitask.py --steps 50000 --n-envs 8 --name v14_smoke
9 |
10 |     # Training largo (6M steps, 16 envs) – lo lanza el usuario cuando decida:
11 |     python Scripts/train_multitask.py --steps 6000000 --n-envs 16 --n-steps 512 \
12 |         --batch-size 256 --n-epochs 10 --net-arch "128,128" --ent-coef 0.005 \
13 |         --learning-rate 3e-4 --name ppo_dum_v14
14 | """
15 |
16 | import argparse
17 | import sys
18 | import time
19 | from pathlib import Path
20 | from datetime import datetime, timedelta
21 |
22 | sys.path.insert(0, str(Path(__file__).resolve().parent))
23 |
24 | # Limitar threads de PyTorch para no pelear con los workers de MuJoCo.
25 | import torch
26 | torch.set_num_threads(2)
27 |
28 | import numpy as np
29 | from stable_baselines3 import PPO
30 | from stable_baselines3.common.vec_env import DummyVecEnv, SubprocVecEnv
31 | from stable_baselines3.common.callbacks import CheckpointCallback
32 | from stable_baselines3.common.monitor import Monitor
33 |
34 | from rl_env_multitask import DUMMultitaskEnv
35 |
36 | RUNS_DIR = Path(__file__).resolve().parents[1] / "runs"
37 | V13_FINAL = RUNS_DIR / "ppo_dum_v13" / "final.zip"
38 |
39 |
40 | def make_env(seed=0, log_dir=None):
41 |     def _init():
42 |         env = DUMMultitaskEnv()
43 |         env.reset(seed=seed)
44 |         if log_dir is not None:
45 |             env = Monitor(env, str(log_dir / f"monitor_{seed}.csv"))
46 |         return env
47 |
48 |     return _init
49 |
50 |
51 | def linear_schedule(initial: float, final: float):
52 |     """SB3 le pasa al schedule un escalar 'progress_remaining' que va de 1.0 a 0.0.
53 |
54 |     Para que el LR baje linealmente de initial a final hay que devolver:
55 |         final + (initial - final) * progress
56 |     """
57 |
58 |     def _f(progress_remaining: float) -> float:
59 |         return final + (initial - final) * progress_remaining
60 |
61 |     return _f
62 |
63 |
64 | def warm_start_from_v13(model: PPO, v13_path: Path) -> dict:
65 |     """Copia los pesos compatibles desde la policy v13 a la policy v14.
66 |
67 |     Devuelve un dict con stats: keys totales, copiadas, skipped (con motivo).
68 |     """
69 |     stats = {"total": 0, "copied": 0, "skipped_missing": [], "skipped_shape": []}
70 |     if not v13_path.exists():
71 |         print(f"[warm-start] No se encontro {v13_path} – entrenando desde cero.")
72 |         stats["skipped_missing"].append(str(v13_path))
73 |         return stats
```

```

74 |
75 | print(f"[warm-start] Cargando v13 desde {v13_path}")
76 | old = PPO.load(str(v13_path), device="cpu")
77 | new_sd = model.policy.state_dict()
78 | old_sd = old.policy.state_dict()
79 |
80 | merged = {}
81 | for k, v_new in new_sd.items():
82 |     stats["total"] += 1
83 |     if k in old_sd:
84 |         v_old = old_sd[k]
85 |         if v_old.shape == v_new.shape:
86 |             merged[k] = v_old
87 |             stats["copied"] += 1
88 |         else:
89 |             merged[k] = v_new
90 |             stats["skipped_shape"].append(
91 |                 f"{k}: new={tuple(v_new.shape)} vs old={tuple(v_old.shape)}"
92 |             )
93 |     else:
94 |         merged[k] = v_new
95 |         stats["skipped_missing"].append(k)
96 |
97 | model.policy.load_state_dict(merged, strict=False)
98 | return stats
99 |
100 |
101 | def main():
102 |     p = argparse.ArgumentParser()
103 |     p.add_argument("--steps", type=int, default=6_000_000)
104 |     p.add_argument("--n-envs", type=int, default=16)
105 |     p.add_argument("--smoke", action="store_true",
106 |                     help="Override: 50k steps, 4 envs.")
107 |     p.add_argument("--resume", type=str, default=None,
108 |                     help="Reanudar desde un checkpoint .zip (no hace warm-start desde v13).")
109 |     p.add_argument("--name", type=str, default=None)
110 |     p.add_argument("--ent-coef", type=float, default=0.005)
111 |     p.add_argument("--learning-rate", type=float, default=3e-4,
112 |                     help="LR inicial del schedule lineal (final = LR/3).")
113 |     p.add_argument("--learning-rate-final", type=float, default=1e-4,
114 |                     help="LR final del schedule lineal.")
115 |     p.add_argument("--n-steps", type=int, default=512)
116 |     p.add_argument("--batch-size", type=int, default=256)
117 |     p.add_argument("--n-epochs", type=int, default=10)
118 |     p.add_argument("--net-arch", type=str, default="128,128",
119 |                     help="MLP de pi y vf. v14 usa 128,128 por defecto para preservar "
120 |                          "el warm-start de v13.")
121 |     p.add_argument("--no-warm-start", action="store_true",
122 |                     help="Entrenar desde cero (saltea la copia desde v13).")
123 |     args = p.parse_args()
124 |
125 |     if args.smoke:
126 |         args.steps = 50_000
127 |         args.n_envs = 4
128 |
129 |     timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
130 |     name = args.name or f"ppo_dum_multitask_{timestamp}"
131 |     run_dir = RUNS_DIR / name
132 |     run_dir.mkdir(parents=True, exist_ok=True)
133 |     print(f"Run dir: {run_dir}")
134 |
135 |     if args.n_envs == 1:
136 |         env = DummyVecEnv([make_env(seed=0, log_dir=run_dir)])
137 |     else:
138 |         env = SubprocVecEnv(
139 |             [make_env(seed=i, log_dir=run_dir) for i in range(args.n_envs)]
140 |         )
141 |
142 |     if args.resume:
143 |         print(f"Resuming from {args.resume}")
144 |         model = PPO.load(args.resume, env=env)
145 |         from_scratch = False
146 |         warm_stats = None
147 |     else:
148 |         net_arch = [int(x) for x in args.net_arch.split(",") if x.strip()]
149 |         lr_sched = linear_schedule(args.learning_rate, args.learning_rate_final)
150 |         model = PPO(
151 |             "MlpPolicy",
152 |             env,
153 |             verbose=1,
154 |             tensorboard_log=str(run_dir / "tb"),
155 |             learning_rate=lr_sched,
156 |             n_steps=args.n_steps,
157 |             batch_size=args.batch_size,
158 |             n_epochs=args.n_epochs,

```

```

159 |         gamma=0.99,
160 |         gae_lambda=0.95,
161 |         clip_range=0.2,
162 |         ent_coef=args.ent_coef,
163 |         vf_coef=0.5,
164 |         policy_kwargs=dict(net_arch=dict(pi=net_arch, vf=net_arch)),
165 |     )
166 |     from_scratch = True
167 |
168 |     if args.no_warm_start:
169 |         print("[warm-start] Saltado por --no-warm-start.")
170 |         warm_stats = None
171 |     else:
172 |         warm_stats = warm_start_from_v13(model, V13_FINAL)
173 |
174 |     checkpoint_cb = CheckpointCallback(
175 |         save_freq=max(args.steps // 10 // args.n_envs, 1),
176 |         save_path=str(run_dir),
177 |         name_prefix="checkpoint",
178 |     )
179 |
180 |     started_at = datetime.now()
181 |     print("=" * 70)
182 |     print(f"INICIO:      {started_at.strftime('%Y-%m-%d %H:%M:%S')}")
183 |     print(f"Run:         {name}")
184 |     print(f"Modo:         {'FROM SCRATCH (red nueva)' if from_scratch else 'RESUMED'}")
185 |     print(f"Steps:         {args.steps:,}")
186 |     print(f"N envs:         {args.n_envs}")
187 |     print(f"n_steps/env:     {args.n_steps} -> rollout total = {args.n_steps * args.n_envs}")
188 |     print(f"batch_size:      {args.batch_size}")
189 |     print(f"n_epochs:        {args.n_epochs}")
190 |     print(f"net_arch:        {args.net_arch}")
191 |     print(f"learning_rate:    {args.learning_rate} -> {args.learning_rate_final} (linear)")
192 |     print(f"ent_coef:         {args.ent_coef}")
193 |     print(f"torch threads:    {torch.get_num_threads()}")
194 |     if warm_stats is not None:
195 |         print(
196 |             f"warm-start v13: copied={warm_stats['copied']}/{warm_stats['total']} "
197 |             f"skipped_shape={len(warm_stats['skipped_shape'])} "
198 |             f"skipped_missing={len(warm_stats['skipped_missing'])}"
199 |         )
200 |         for s in warm_stats["skipped_shape"]:
201 |             print(f"    shape mismatch: {s}")
202 |     print("=" * 70, flush=True)
203 |
204 |     t0 = time.perf_counter()
205 |     model.learn(
206 |         total_timesteps=args.steps,
207 |         callback=checkpoint_cb,
208 |         progress_bar=True,
209 |     )
210 |     elapsed = time.perf_counter() - t0
211 |
212 |     final_path = run_dir / "final.zip"
213 |     model.save(str(final_path))
214 |     finished_at = datetime.now()
215 |
216 |     print("=" * 70)
217 |     print(f"INICIO:      {started_at.strftime('%Y-%m-%d %H:%M:%S')}")
218 |     print(f"FIN:         {finished_at.strftime('%Y-%m-%d %H:%M:%S')}")
219 |     print(f"DURACION:    {timedelta(seconds=int(elapsed))} ({elapsed:.1f} s)")
220 |     print(f"Modelo final: {final_path}")
221 |     print("=" * 70)
222 |
223 |
224 | if __name__ == "__main__":
225 |     main()
226 |

```